

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

BUKKAPATNAM VARSHYARA (1BM24CS075)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Bukkapatnam Varshyara (**1BM24CS075**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	6
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	10
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	12
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	14
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	16
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	18
9	Implement Fractional Knapsack using Greedy technique.	22
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	24
11	Implement "N-Queens Problem" using Backtracking.	26
12	LeetCode Problems	28

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Code:

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 100
4  int graph[MAX][MAX];
5  int visited[MAX];
6  int stack[MAX];
7  int top = -1;
8  int n;
9  void dfs(int v)
10 {
11     visited[v] = 1;
12     for (int i = 0; i < n; i++)
13     {
14         if (graph[v][i] && !visited[i])
15         {
16             dfs(i);
17         }
18     }
19     stack[++top] = v;
20 }
21 void topologicalSort()
22 {
23     for (int i = 0; i < n; i++)
24     {
25         if (!visited[i])
26         {
27             dfs(i);
28         }
29     }
30     printf("\nTopological Ordering:\n");
31     while (top != -1)
32     {
33         printf("%d ", stack[top--]);
34     }
35     printf("\n");
36 }
37 int main()
38 {
39     clock_t start, end;
40     double cpu_time_used;
41     printf("Enter number of vertices: ");
42     scanf("%d", &n);
43     printf("Enter adjacency matrix:\n");
44     for (int i = 0; i < n; i++)
45     {
46         for (int j = 0; j < n; j++)
47         {
48             scanf("%d", &graph[i][j]);
49         }
50     }
51     for (int i = 0; i < n; i++)
52     {
53         visited[i] = 0;
54     }
55     start = clock();
56     topologicalSort();
57     end = clock();
58     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
59     printf("\nExecution Time = %lf seconds\n", cpu_time_used);
60     return 0;
61 }
```

Output:

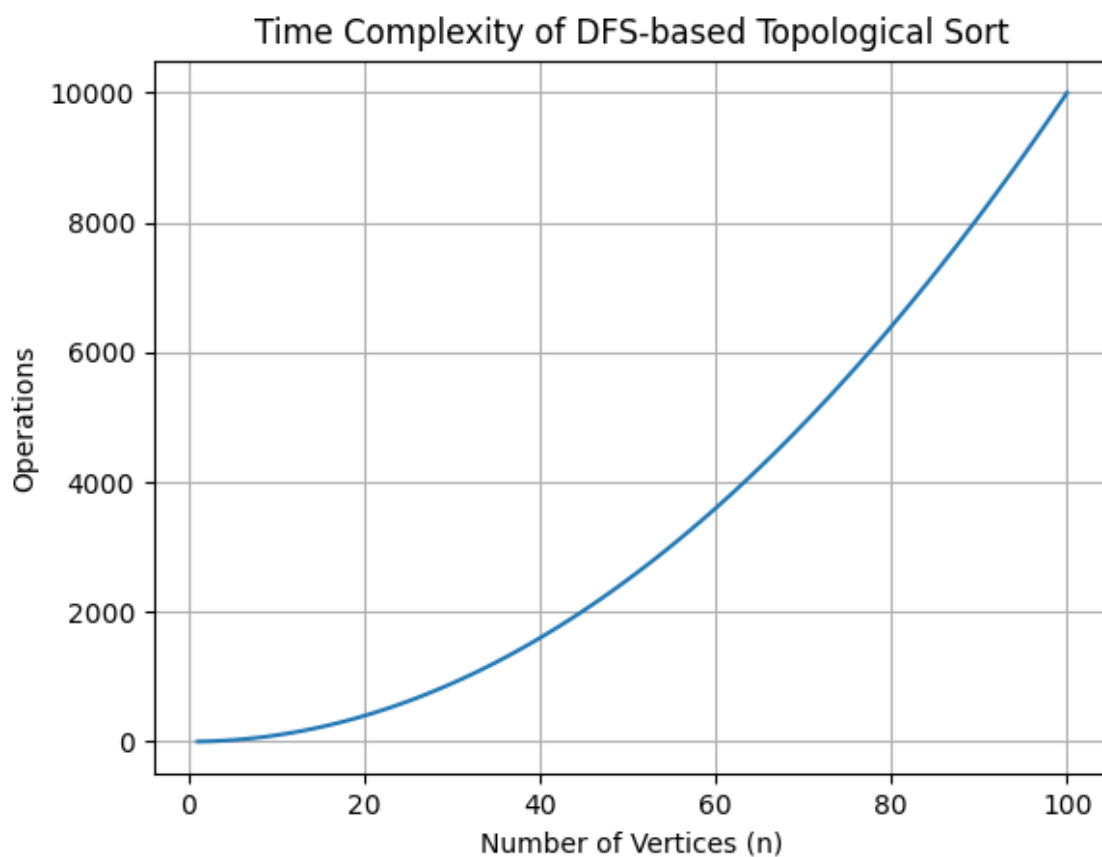
```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological Ordering:
0 2 1 3

Execution Time = 0.002000 seconds
```

Graph:

Time Complexity : $O(n^2)$



Lab Program 2:

```
1  #include <stdio.h>
2  #include <time.h>
3  int NN;
4  int p[100], pi[100];
5  int dir[100];
6  void PrintPerm()
7  {
8      for (int i = 1; i <= NN; i++)
9          printf("%d ", p[i]);
10     printf("\n");
11 }
12 void Move(int x, int d)
13 {
14     int z;
15     z = p[pi[x] + d];
16     p[pi[x]] = z;
17     p[pi[x] + d] = x;
18     pi[z] = pi[x];
19     pi[x] = pi[x] + d;
20 }
21 void Perm(int n)
22 {
23     if (n > NN)
24         PrintPerm();
25     else
26     {
27         Perm(n + 1);
28         for (int i = 1; i <= n - 1; i++)
29         {
30             Move(n, dir[n]);
31             Perm(n + 1);
32         }
33         dir[n] = -dir[n];
34     }
35 }
36 int main()
37 {
38     clock_t start, end;
39     double execution_time;
40     printf("Enter n: ");
41     scanf("%d", &NN);
42     for (int i = 1; i <= NN; i++)
43     {
44         p[i] = i;
45         pi[i] = i;
46         dir[i] = -1;
47     }
48     printf("\nPermutations:\n");
49     start = clock();
50     Perm(1);
51     end = clock();
52     execution_time =
53         (double) (end - start) / CLOCKS_PER_SEC;
54     printf("\nExecution Time = %lf seconds\n",
55         execution_time);
56     return 0;
57 }
```

Output:

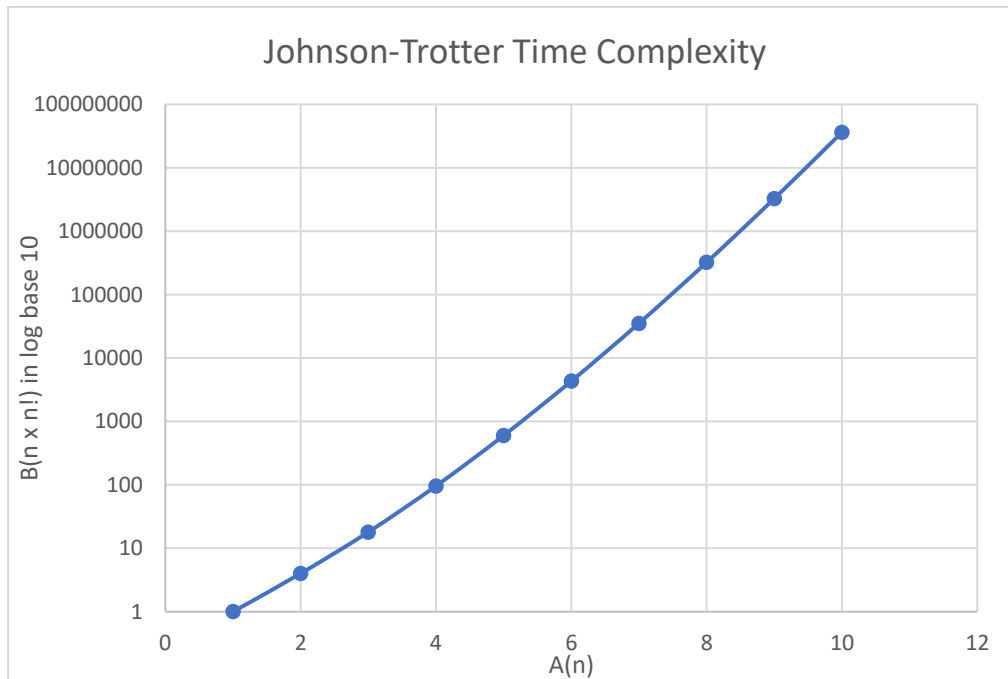
```
Enter n: 3

Permutations:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Execution Time = 0.003000 seconds
```

Graph:

Time Complexity : $O(n \times n!)$



Lab Program 3:

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 100
4  int main()
5  {
6      int low, mid, high;
7      int a[MAX], b[MAX];
8      printf("Enter low index: ");
9      scanf("%d", &low);
10     printf("Enter mid index: ");
11     scanf("%d", &mid);
12     printf("Enter high index: ");
13     scanf("%d", &high);
14     printf("\nEnter %d sorted elements:\n", high + 1);
15     for (int i = low; i <= high; i++)
16         scanf("%d", &a[i]);
17     clock_t start, end;
18     double execution_time;
19     start = clock();
20     int i = low;
21     int j = mid + 1;
22     int k = low;
23     while (i <= mid && j <= high)
24     {
25         if (a[i] <= a[j])
26         {
27             b[k] = a[i];
28             i++;
29         }
30         else
31         {
32             b[k] = a[j];
33             j++;
34         }
35         k++;
36     }
37     while (i <= mid)
38     {
39         b[k] = a[i];
40         i++;
41         k++;
42     }
43     while (j <= high)
44     {
45         b[k] = a[j];
46         j++;
47         k++;
48     }
49     end = clock();
50     execution_time =
51         ((double)(end - start)) / CLOCKS_PER_SEC;
52     printf("\nMerged Array:\n");
53     for (k = low; k <= high; k++)
54         printf("%d ", b[k]);
55     printf("\n");
56     printf("\nExecution Time = %lf seconds\n",
57         execution_time);
58     return 0;
59 }
```


Output:

```
Enter low index: 0
Enter mid index: 3
Enter high index: 7

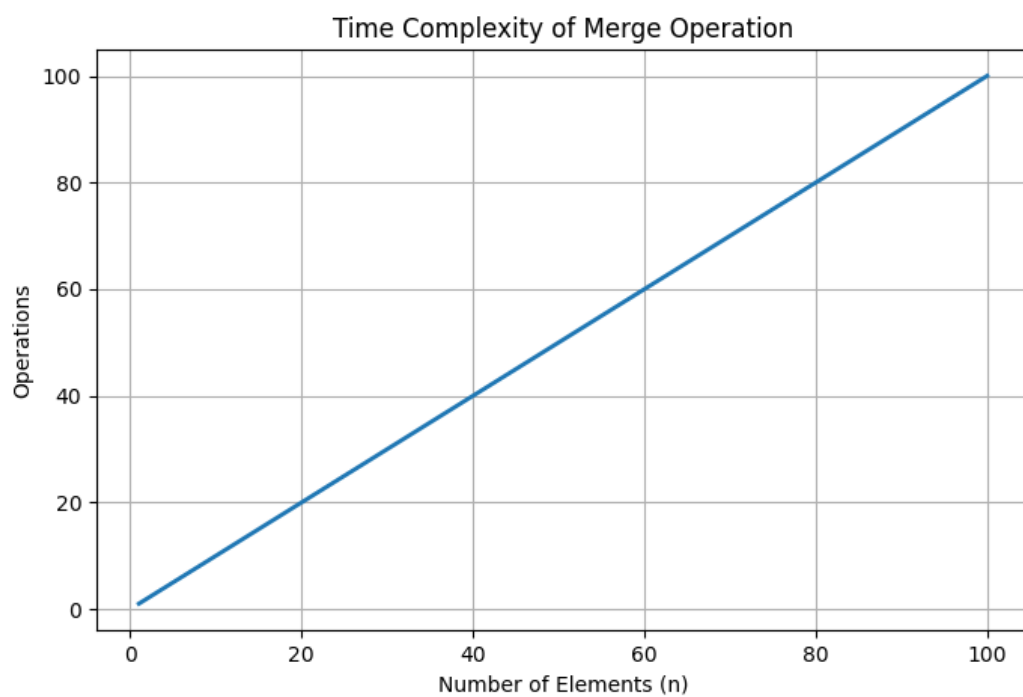
Enter 8 sorted elements:
10 20 30 40 15 25 35 45

Merged Array:
10 15 20 25 30 35 40 45

Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : $O(n)$



Lab Program 4:

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 100
4  int partition(int a[], int low, int high)
5  {
6      int pivot = a[low];
7      int i = low + 1;
8      int j = high;
9      int temp;
10     while (1)
11     {
12         while (i <= high && a[i] <= pivot)
13             i++;
14         while (a[j] > pivot)
15             j--;
16         if (i < j)
17         {
18             temp = a[i];
19             a[i] = a[j];
20             a[j] = temp;
21         }
22         else
23         {
24             temp = a[low];
25             a[low] = a[j];
26             a[j] = temp;
27             return j;
28         }
29     }
30 }
31 void quicksort(int a[], int low, int high)
32 {
33     if (low < high)
34     {
35         int pivotIndex = partition(a, low, high);
36         quicksort(a, low, pivotIndex - 1);
37         quicksort(a, pivotIndex + 1, high);
38     }
39 }
40 int main()
41 {
42     int a[MAX];
43     int n;
44     clock_t start, end;
45     double execution_time;
46     printf("Enter number of elements: ");
47     scanf("%d", &n);
48     printf("Enter elements:\n");
49     for (int i = 0; i < n; i++)
50         scanf("%d", &a[i]);
51     start = clock();
52     quicksort(a, 0, n - 1);
53     end = clock();
54     execution_time =
55         (double)(end - start) / CLOCKS_PER_SEC;
56     printf("\nSorted Array:\n");
57     for (int i = 0; i < n; i++)
58         printf("%d ", a[i]);
59     printf("\n");
60     printf("\nExecution Time = %lf seconds\n",
61         execution_time);
62     return 0;
63 }
```

Output:

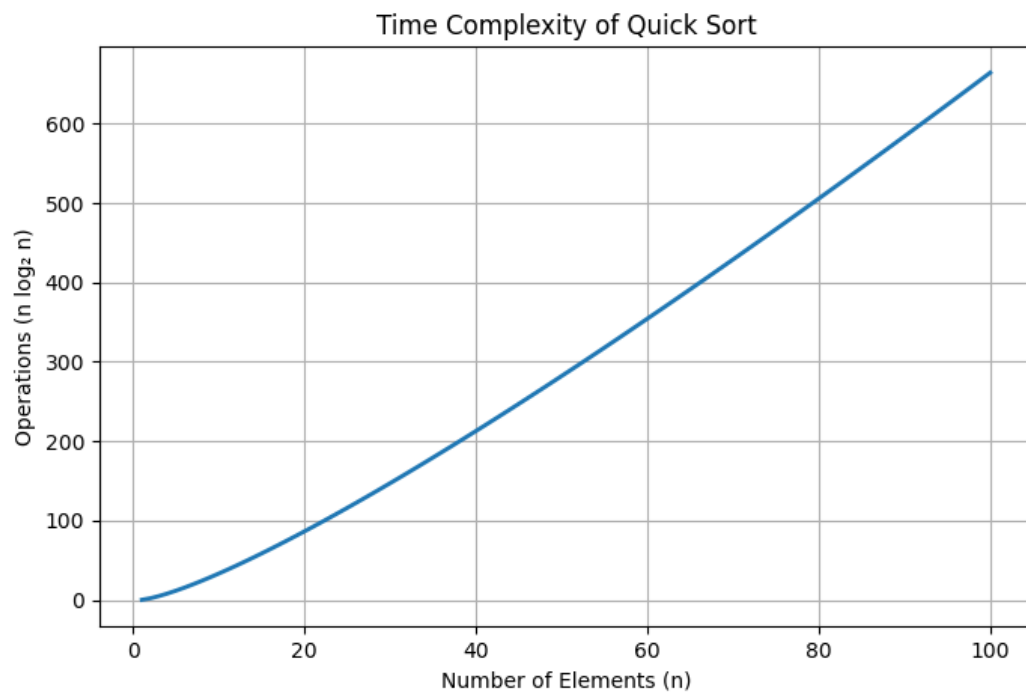
```
Enter number of elements: 10
Enter elements:
78 3 44 16 9 33 28 14 99 88

Sorted Array:
3 9 14 16 28 33 44 78 88 99

Execution Time = 0.000000 seconds
```

Graph:

Time complexity : $O(n \log n)$



Lab Program 5:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  void heapify(int a[], int n, int i)
5  {
6      int c, item;
7      int p=i;
8      item=a[p];
9      c=2*p+1;
10     while(c<=n-1){
11         if(c+1<=n-1){
12             if(a[c]<a[c+1]) c++;
13         }
14         if(item<a[c]){
15             a[p]=a[c];
16             p=c;
17             c=2*p+1;
18         }
19         else break;
20     }
21     a[p]=item;
22 }
23 void buildheap(int a[],int n){
24     int i;
25     for(i=(n-1)/2;i>=0;i--){
26         heapify(a,n,i);
27     }
28 }
29 void heapSort(int a[], int n)
30 {
31     int i, temp;
32     buildheap(a,n);
33     for(i=n-1;i>0;i--){
34         temp=a[0];
35         a[0]=a[i];
36         a[i]=temp;
37         heapify(a,i,0);
38     }
39 }
40 int main()
41 {
42     int n;
43     clock_t start, end;
44     double cpu_time;
45     printf("Enter number of elements: ");
46     scanf("%d", &n);
47     int a[n];
48     printf("Enter elements:\n");
49     for (int i = 0; i < n; i++)
50         scanf("%d", &a[i]);
51     start = clock();
52     heapSort(a, n);
53     end = clock();
54     cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
55     printf("\nSorted elements are:\n");
56     for (int i = 0; i < n; i++)
57         printf("%d ", a[i]);
58     printf("\n\nTime taken = %f seconds\n", cpu_time);
59     return 0;
60 }
```

Output:

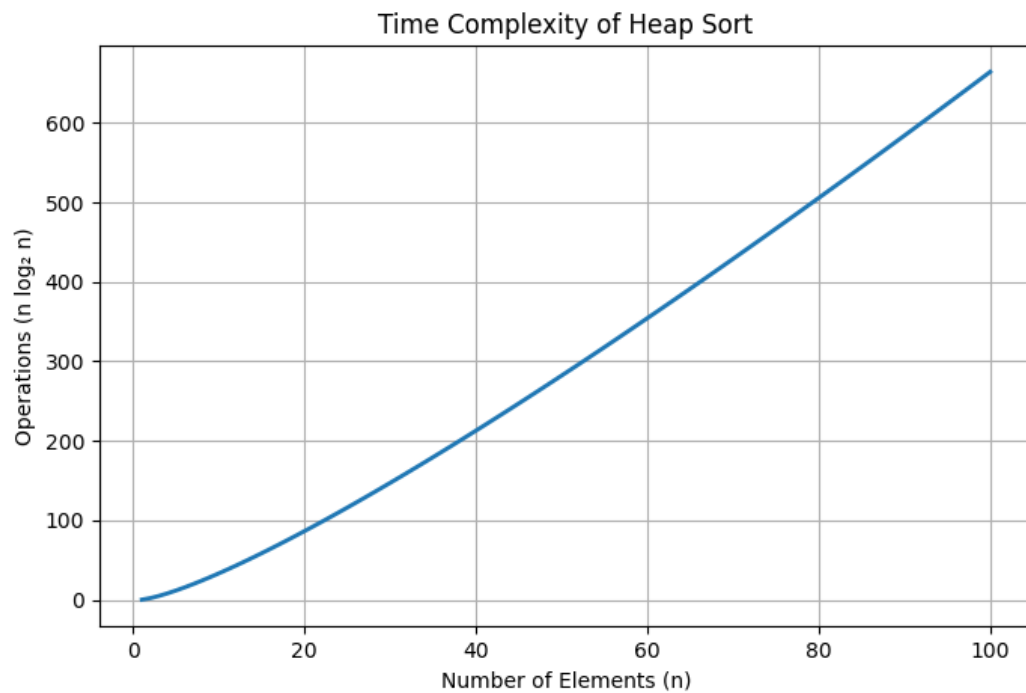
```
Enter number of elements: 6
Enter elements:
4 2 6 1 8 3

Sorted elements are:
1 2 3 4 6 8

Time taken = 0.000000 seconds
```

Graph:

Time Complexity : $O(n \log n)$



Lab Program 6:

```
1  #include <stdio.h>
2  #include <time.h>
3  int max(int a, int b) {
4      return (a > b) ? a : b;
5  }
6  // 0/1 Knapsack using Dynamic Programming
7  int knapsack(int W, int wt[], int pt[], int n) {
8      int i, w;
9      int K[n + 1][W + 1];
10     for (i = 0; i <= n; i++) {
11         for (w = 0; w <= W; w++) {
12             if (i == 0 || w == 0)
13                 K[i][w] = 0;
14             else if (wt[i - 1] <= w)
15                 K[i][w] = max(pt[i - 1] + K[i - 1][w - wt[i - 1]],
16                               K[i - 1][w]);
17             else
18                 K[i][w] = K[i - 1][w];
19         }
20     }
21     return K[n][W];
22 }
23 int main() {
24     int n, W;
25     printf("Enter number of items: ");
26     scanf("%d", &n);
27     int pt[n], wt[n];
28     printf("Enter weights of items:\n");
29     for (int i = 0; i < n; i++)
30         scanf("%d", &wt[i]);
31     printf("Enter profit of items:\n");
32     for (int i = 0; i < n; i++)
33         scanf("%d", &pt[i]);
34     printf("Enter capacity of knapsack: ");
35     scanf("%d", &W);
36     clock_t start, end;
37
38     start = clock(); // Start time
39     int result = knapsack(W, wt, pt, n);
40     end = clock(); // End time
41     double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
42     printf("\nMaximum Profit = %d\n", result);
43     printf("Execution Time = %f seconds\n", time_taken);
44     return 0;
45 }
```

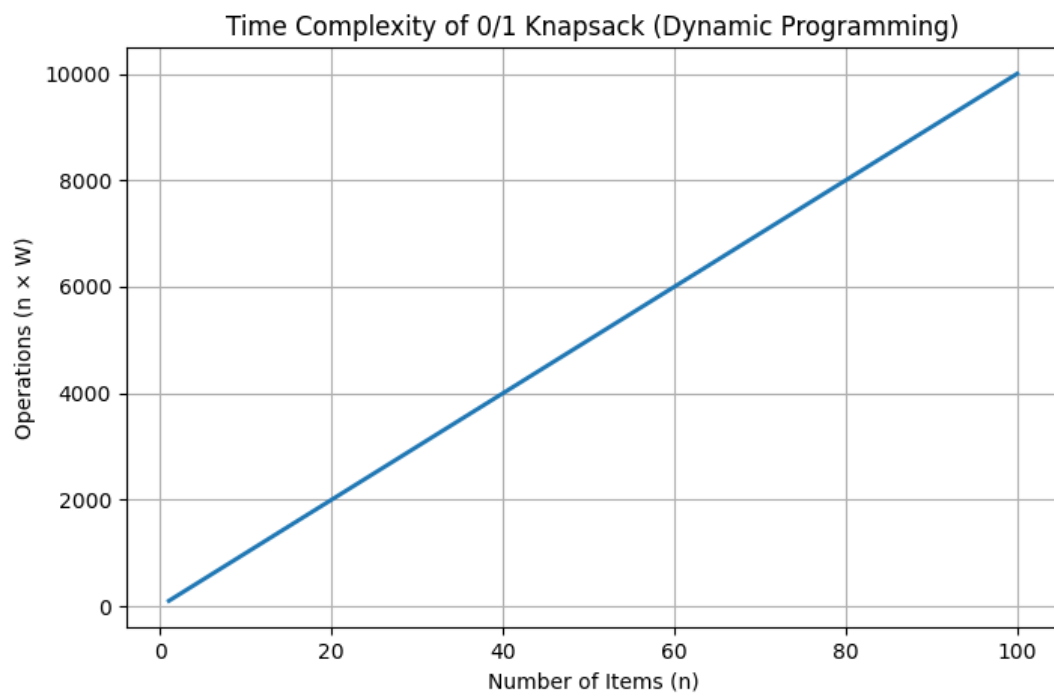
Output:

```
Enter number of items: 3
Enter weights of items:
2 3 5
Enter profit of items:
20 10 40
Enter capacity of knapsack: 6

Maximum Profit = 40
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : $O(nW)$



Lab Program 7:

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 20
4  #define INF 9999
5  void floyds(int n, int cost[MAX][MAX], int A[MAX][MAX]) {
6      int i, j, k;
7      for(i = 0; i < n; i++) {
8          for(j = 0; j < n; j++) {
9              A[i][j] = cost[i][j];
10             }
11         }
12         for(k = 0; k < n; k++) {
13             for(i = 0; i < n; i++) {
14                 for(j = 0; j < n; j++) {
15                     if(A[i][k] + A[k][j] < A[i][j]) {
16                         A[i][j] = A[i][k] + A[k][j];
17                     }
18                 }
19             }
20         }
21     }
22 int main() {
23     int n;
24     int cost[MAX][MAX], A[MAX][MAX];
25     int i, j;
26     clock_t start, end;
27     double cpu_time_used;
28     printf("Enter number of vertices: ");
29     scanf("%d", &n);
30     printf("Enter the cost matrix:\n");
31     printf("(Enter %d for INF / no direct edge)\n", INF);
32     for(i = 0; i < n; i++) {
33         for(j = 0; j < n; j++) {
34             scanf("%d", &cost[i][j]);
35         }
36     }
37     start = clock();
38     floyds(n, cost, A);
39     end = clock();
40     cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
41     printf("\nShortest Distance Matrix:\n");
42     for(i = 0; i < n; i++) {
43         for(j = 0; j < n; j++) {
44             if(A[i][j] == INF)
45                 printf("INF\t");
46             else
47                 printf("%d\t", A[i][j]);
48         }
49         printf("\n");
50     }
51     printf("\nExecution Time: %f seconds\n", cpu_time_used);
52     return 0;
53 }
```


Output:

```
Enter number of vertices: 4
Enter the cost matrix:
(Enter 9999 for INF / no direct edge)
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0
```

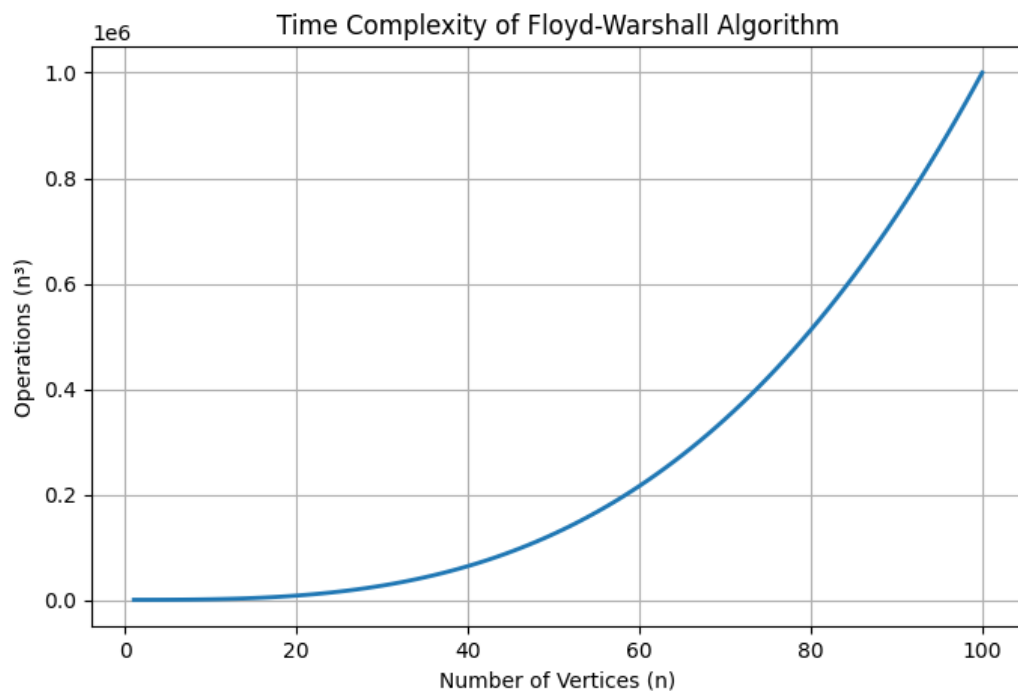
Shortest Distance Matrix:

```
0      5      8      9
INF    0      3      4
INF    INF    0      1
INF    INF    INF    0
```

Execution Time: 0.000000 seconds

Graph:

Time Complexity : $O(n^3)$



Lab Program 8:

a)

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 20
4  #define INF 9999
5  int main()
6  {
7      int n;
8      int cost[MAX][MAX];
9      int visited[MAX] = {0};
10     int minCost = 0;
11     int edges = 0;
12     printf("Enter number of vertices: ");
13     scanf("%d", &n);
14     printf("Enter Cost Adjacency Matrix:\n");
15     printf("(Enter %d for no edge)\n", INF);
16     for(int i = 0; i < n; i++)
17     {
18         for(int j = 0; j < n; j++)
19             scanf("%d", &cost[i][j]);
20     }
21     clock_t start, end;
22     double execution_time;
23     start = clock();
24     visited[0] = 1;
25     printf("\nEdges in MST:\n");
26     while(edges < n - 1)
27     {
28         int min = INF;
29         int u = -1, v = -1;
30         for(int i = 0; i < n; i++)
31         {
32             if(visited[i])
33             {
34                 for(int j = 0; j < n; j++)
35                 {
36                     if(!visited[j] && cost[i][j] < min)
37                     {
38                         min = cost[i][j];
39                         u = i;
40                         v = j;
41                     }
42                 }
43             }
44         }
45         printf("%d -> %d Cost = %d\n", u, v, min);
46         minCost += min;
47         visited[v] = 1;
48         edges++;
49     }
50     end = clock();
51     execution_time = (double)(end - start) / CLOCKS_PER_SEC;
52     printf("\nMinimum Cost = %d\n", minCost);
53     printf("Execution Time = %lf seconds\n", execution_time);
54     return 0;
55 }
```

Output:

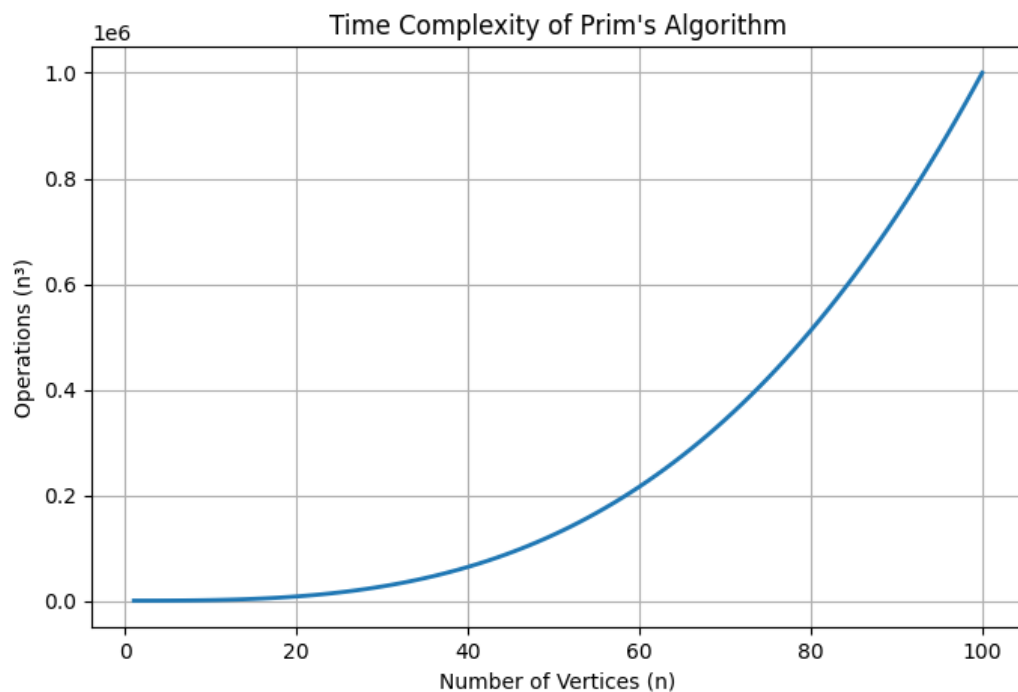
```
Enter number of vertices: 5
Enter Cost Adjacency Matrix:
(Enter 9999 for no edge)
9999 12 41 9999 14
12 9999 30 25 9999
41 30 9999 12 11
9999 25 12 9999 6
14 9999 11 6 9999

Edges in MST:
0 -> 1 Cost = 12
0 -> 4 Cost = 14
4 -> 3 Cost = 6
4 -> 2 Cost = 11

Minimum Cost = 43
Execution Time = 0.001000 seconds
```

Graph:

Time Complexity : $O(n^3)$



b)

```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 100
4  int parent[MAX];
5  int find(int v)
6  {
7      while(parent[v] != 0)
8          v = parent[v];
9      return v;
10 }
11 void unite(int u, int v)
12 {
13     parent[v] = u;
14 }
15 int main()
16 {
17     int n, e;
18     int u[MAX], v[MAX], w[MAX];
19     printf("Enter number of vertices: ");
20     scanf("%d", &n);
21     printf("Enter number of edges: ");
22     scanf("%d", &e);
23     printf("\nEnter edges (source destination weight):\n");
24     for(int i = 0; i < e; i++)
25     {
26         scanf("%d %d %d",
27             &u[i],
28             &v[i],
29             &w[i]);
30     }
31     clock_t start, end;
32     double execution_time;
33     start = clock();
34     for(int i = 0; i < e - 1; i++)
35     {
36         for(int j = 0; j < e - i - 1; j++)
37         {
38             if(w[j] > w[j + 1])
39             {
40                 int temp;
41                 temp = w[j];
42                 w[j] = w[j + 1];
43                 w[j + 1] = temp;
44                 temp = u[j];
45                 u[j] = u[j + 1];
46                 u[j + 1] = temp;
47                 temp = v[j];
48                 v[j] = v[j + 1];
49                 v[j + 1] = temp;
50             }
51         }
52     }
53     int minCost = 0;
54     int count = 0;
55     printf("\nEdges in MST:\n");
56     for(int i = 0; i < e && count < n - 1; i++)
57     {
58         int p1 = find(u[i]);
59         int p2 = find(v[i]);
60         if(p1 != p2)
61         {
62             printf("%d -- %d Cost = %d\n",
63                 u[i], v[i], w[i]);
64             minCost += w[i];
65             unite(p1, p2);
66             count++;
67         }
68     }
69     end = clock();
70     execution_time = (double)(end - start) / CLOCKS_PER_SEC;
71     printf("\nMinimum Cost = %d\n", minCost);
72     printf("Execution Time = %lf seconds\n", execution_time);
```

Output:

```
Enter number of vertices: 4
Enter number of edges: 5

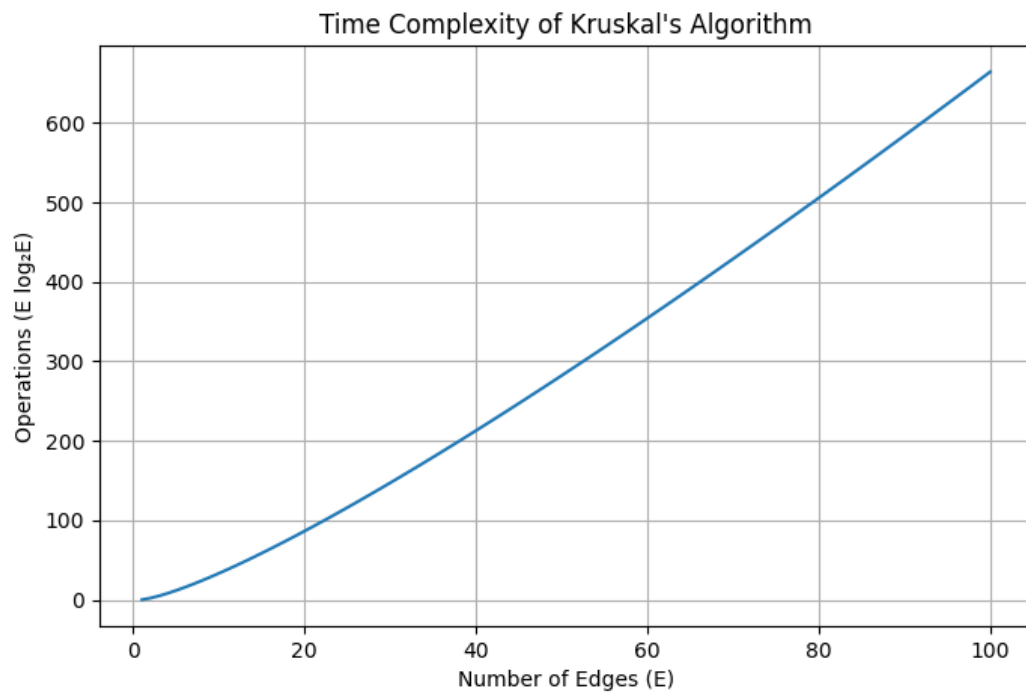
Enter edges (source destination weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in MST:
2 -- 3 Cost = 4
0 -- 3 Cost = 5
0 -- 2 Cost = 6

Minimum Cost = 15
Execution Time = 0.001000 seconds
```

Graph:

Time Complexity : $O(n \log n)$



Lab Program 9:

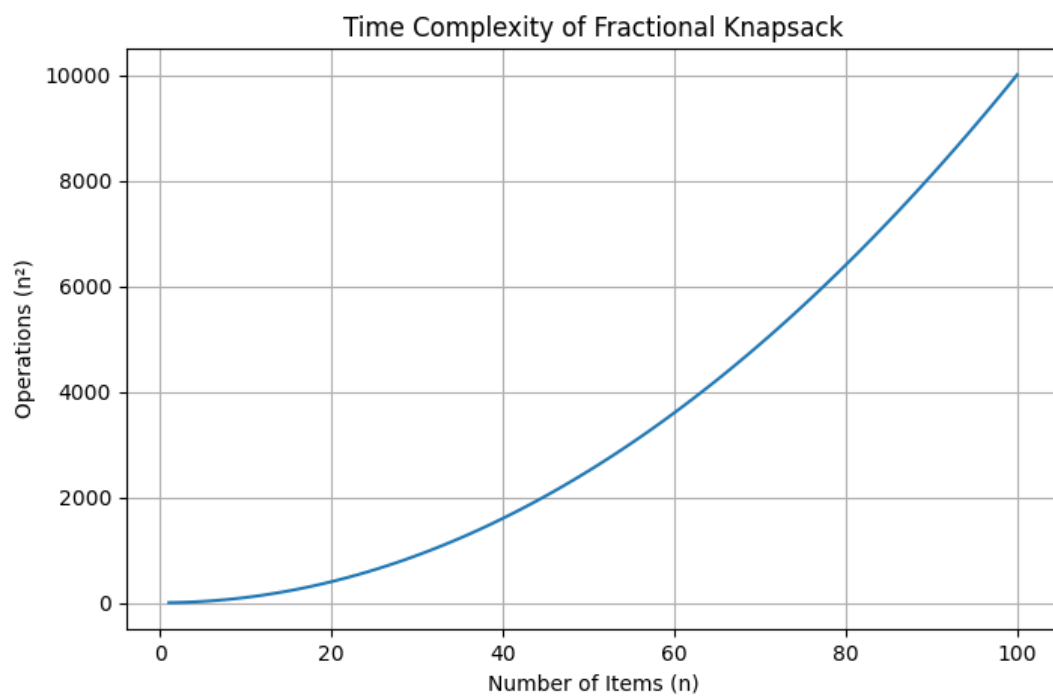
```
1  #include <stdio.h>
2  #include <time.h>
3  int main() {
4      int n, W;
5      printf("Enter number of items: ");
6      scanf("%d", &n);
7      int value[n], weight[n];
8      float ratio[n];
9      printf("Enter value and weight:\n");
10     for(int i = 0; i < n; i++) {
11         scanf("%d %d", &value[i], &weight[i]);
12         ratio[i] = (float)value[i] / weight[i];
13     }
14     printf("Enter capacity: ");
15     scanf("%d", &W);
16     clock_t start = clock();
17     for(int i = 0; i < n - 1; i++) {
18         for(int j = 0; j < n - i - 1; j++) {
19             if(ratio[j] < ratio[j + 1]) {
20                 float temp = ratio[j];
21                 ratio[j] = ratio[j + 1];
22                 ratio[j + 1] = temp;
23                 int t1 = value[j];
24                 value[j] = value[j + 1];
25                 value[j + 1] = t1;
26                 int t2 = weight[j];
27                 weight[j] = weight[j + 1];
28                 weight[j + 1] = t2;
29             }
30         }
31     }
32     float totalProfit = 0.0;
33     for(int i = 0; i < n; i++) {
34         if(weight[i] <= W) {
35             totalProfit += value[i];
36             W -= weight[i];
37         } else {
38             totalProfit += value[i] * ((float)W / weight[i]);
39             break;
40         }
41     }
42     clock_t end = clock();
43     double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
44     printf("Maximum Profit = %.2f\n", totalProfit);
45     printf("Execution Time = %f seconds\n", time_taken);
46     return 0;
47 }
```

Output:

```
Enter number of items: 5
Enter value and weight:
200 5
500 10
300 1
600 4
400 3
Enter capacity: 6
Maximum Profit = 1033.33
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : $O(n^2)$



Lab Program 10:

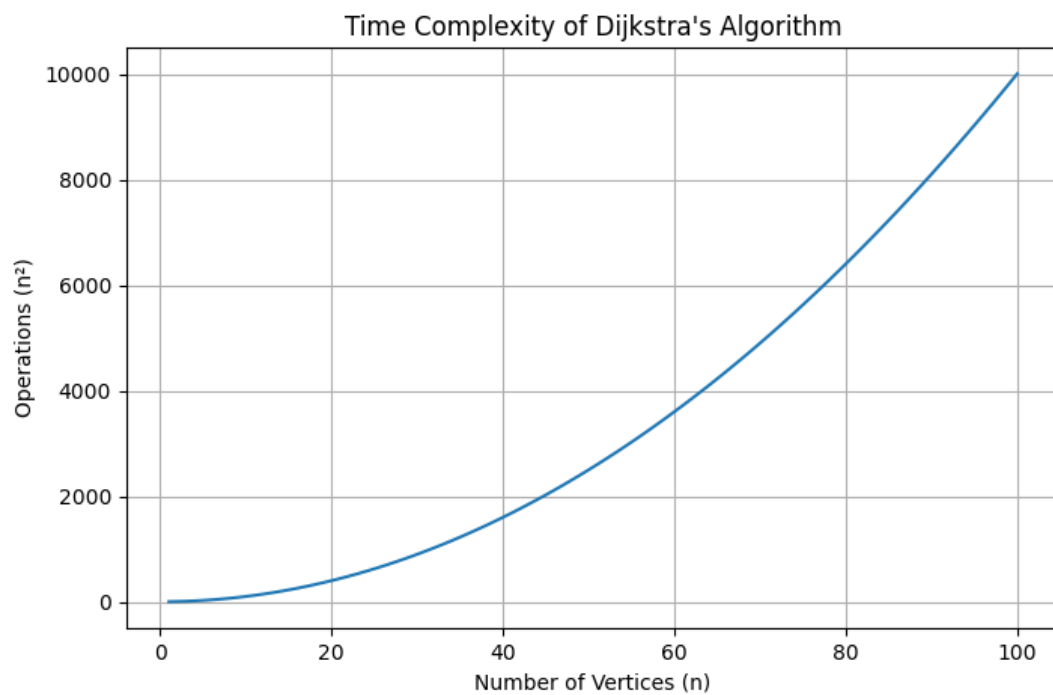
```
1  #include <stdio.h>
2  #include <time.h>
3  #define MAX 100
4  #define INF 99999
5  int minDistance(int dist[], int visited[], int n) {
6      int min = INF, min_index;
7      for (int i = 0; i < n; i++) {
8          if (visited[i] == 0 && dist[i] <= min) {
9              min = dist[i];
10             min_index = i;
11         }
12     }
13     return min_index;
14 }
15 void dijkstra(int graph[MAX][MAX], int n, int src) {
16     int dist[MAX], visited[MAX];
17     for (int i = 0; i < n; i++) {
18         dist[i] = INF;
19         visited[i] = 0;
20     }
21     dist[src] = 0;
22     clock_t start = clock();
23     for (int count = 0; count < n - 1; count++) {
24         int u = minDistance(dist, visited, n);
25         visited[u] = 1;
26         for (int v = 0; v < n; v++) {
27             if (!visited[v] && graph[u][v] != 0 &&
28                 dist[u] != INF &&
29                 dist[u] + graph[u][v] < dist[v]) {
30                 dist[v] = dist[u] + graph[u][v];
31             }
32         }
33     }
34     clock_t end = clock();
35     double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
36     printf("Vertex\tDistance from Source\n");
37
38     for (int i = 0; i < n; i++) {
39         printf("%d\t%d\n", i, dist[i]);
40     }
41     printf("Execution Time = %f seconds\n", time_taken);
42 }
43 int main() {
44     int n, graph[MAX][MAX], src;
45     printf("Enter number of vertices: ");
46     scanf("%d", &n);
47     printf("Enter adjacency matrix:\n");
48     for (int i = 0; i < n; i++) {
49         for (int j = 0; j < n; j++) {
50             scanf("%d", &graph[i][j]);
51         }
52     }
53     printf("Enter source vertex: ");
54     scanf("%d", &src);
55     dijkstra(graph, n, src);
56     return 0;
57 }
```


Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 10 0 30
10 0 50 0
0 50 0 20
30 0 20 0
Enter source vertex: 1
Vertex Distance from Source
0      10
1       0
2      50
3      40
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : $O(n^2)$



Lab Program 11:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define MAX 20
5  int board[MAX];
6  int solutionCount = 0;
7  int isSafe(int row, int col)
8  {
9      int i;
10     for (i = 0; i < row; i++)
11     {
12         if (board[i] == col)
13             return 0;
14         if (abs(board[i] - col) == abs(i - row))
15             return 0;
16     }
17     return 1;
18 }
19 void printBoard(int n)
20 {
21     int i, j;
22     printf("\nSolution %d:\n\n", ++solutionCount);
23     for (i = 0; i < n; i++)
24     {
25         for (j = 0; j < n; j++)
26         {
27             if (board[i] == j)
28                 printf(" Q ");
29             else
30                 printf(" . ");
31         }
32         printf("\n");
33     }
34 }
35 void solveNQueens(int row, int n)
36 {
37     int col;
38     if (row == n)
39     {
40         printBoard(n);
41         return;
42     }
43     for (col = 0; col < n; col++)
44     {
45         if (isSafe(row, col))
46         {
47             board[row] = col;
48             solveNQueens(row + 1, n);
49         }
50     }
51 }
52 int main()
53 {
54     int n;
55     clock_t start, end;
56     double cpu_time_used;
57     printf("Enter number of queens: ");
58     scanf("%d", &n);
59     start = clock();
60     solveNQueens(0, n);
61     end = clock();
62     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
63     printf("\nTotal Solutions = %d\n", solutionCount);
64     printf("Execution Time = %f seconds\n", cpu_time_used);
65     return 0;
66 }
```

Output:

```
Enter number of queens: 4

Solution 1:

. Q . .
. . . Q
Q . . .
. . Q .

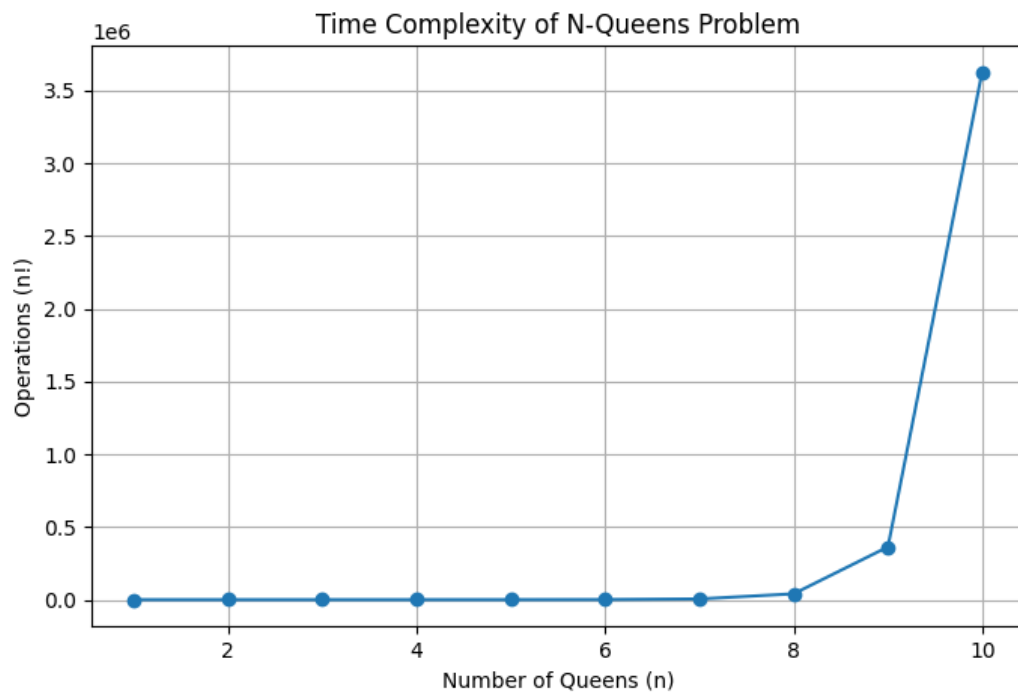
Solution 2:

. . Q .
Q . . .
. . . Q
. Q . .

Total Solutions = 2
Execution Time = 0.004000 seconds
```

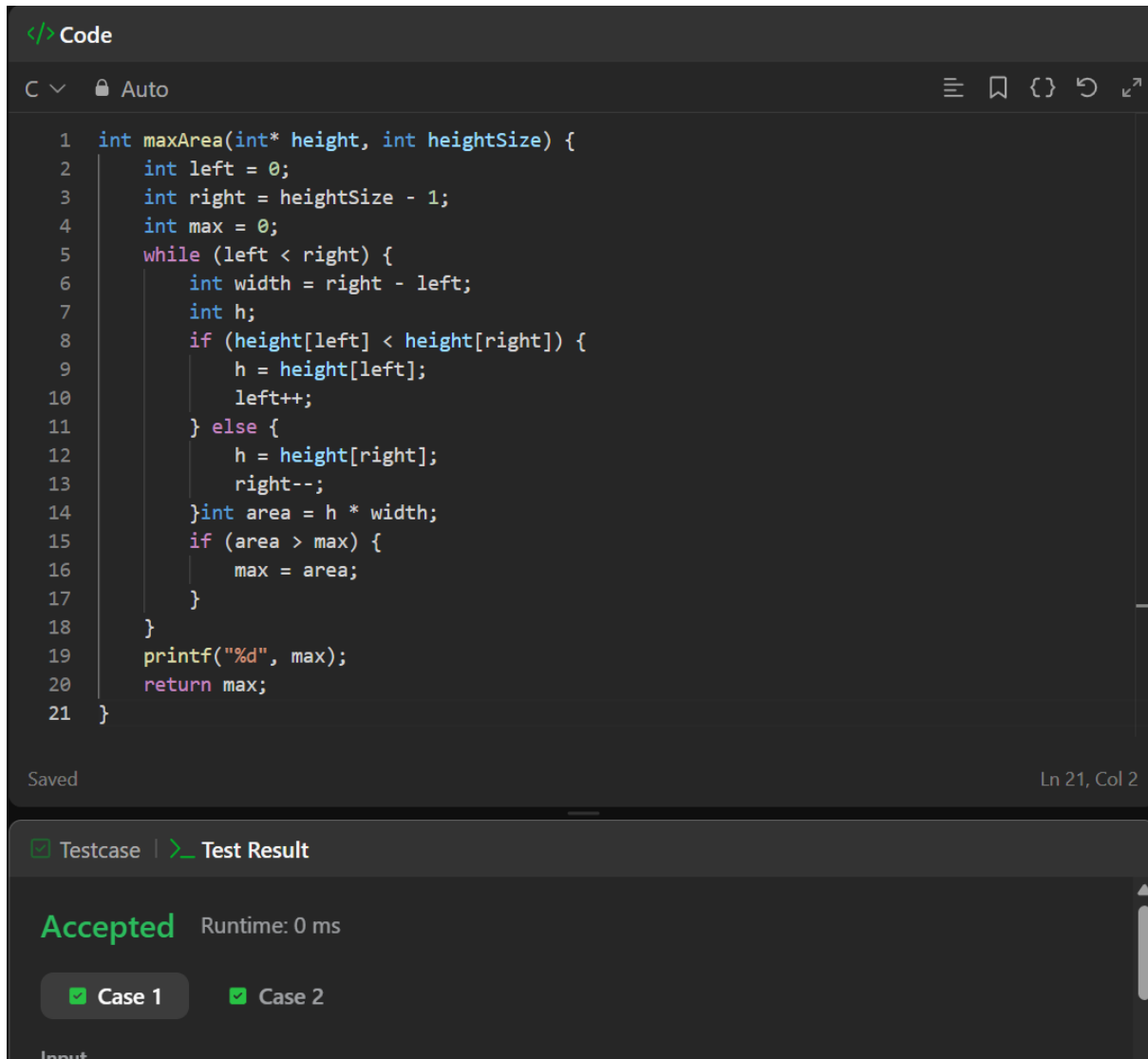
Graph:

Time Complexity : $O(n!)$



LeetCode Problems:

11. Container with most water



```
1 int maxArea(int* height, int heightSize) {
2     int left = 0;
3     int right = heightSize - 1;
4     int max = 0;
5     while (left < right) {
6         int width = right - left;
7         int h;
8         if (height[left] < height[right]) {
9             h = height[left];
10            left++;
11        } else {
12            h = height[right];
13            right--;
14        }int area = h * width;
15        if (area > max) {
16            max = area;
17        }
18    }
19    printf("%d", max);
20    return max;
21 }
```

Saved Ln 21, Col 2

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

34. Find first and last position of element in sorted array

```
</>Code
C v Auto
1 int* searchRange(int* nums, int numsSize, int target, int* returnSize)
2 {
3     int *retArr= (int *) malloc (2*sizeof(int));
4     *returnSize=2;
5     retArr[0]=-1;
6     retArr[1]=-1;
7     for(int i=0;i<numsSize;i++)
8     {
9         if(nums[i]==target)
10        {
11            retArr[0]=i;
12            while(i+1<numsSize&&nums[i+1]==target)
13            {
14                i++;
15            }
16            retArr[1]=i;
17            break;
18        }
19    }
20    return retArr;
21 }
```

Saved Ln 21, Col

☒ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

268. Missing number

```
</> Code
C ▾ 🔒 Auto
☰ 📖 {} ↶

1  int missingNumber(int* nums, int numsSize) {
2      int n = numsSize;
3      int expectedSum = n * (n + 1) / 2;
4      int actualSum = 0;
5      for (int i = 0; i < numsSize; i++) {
6          actualSum += nums[i];
7      }
8      return expectedSum - actualSum;
9  }
```

Saved Ln 9, Col

☒ Testcase | **> Test Result**

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

1636. Sort array by increasing frequency

```
</> Code
C v Auto
1 int freq[201];
2 int compare(const void *a, const void *b)
3 {
4     int x = *(int *)a;
5     int y = *(int *)b;
6     if (freq[x + 100] != freq[y + 100])
7         return freq[x + 100] - freq[y + 100];
8     return y - x;
9 }
10 int* frequencySort(int* nums, int numsSize, int* returnSize)
11 {
12     for (int i = 0; i < 201; i++)
13         freq[i] = 0;
14     for (int i = 0; i < numsSize; i++)
15         freq[nums[i] + 100]++;
16     qsort(nums, numsSize, sizeof(int), compare);
17     *returnSize = numsSize;
18     return nums;
19 }
Saved Ln 17, Col
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
```

207. Course Schedule

```
</> Code
C v Auto
☰ 📖 {} ↺

1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
  prerequisitesColSize) {
2     int indegree[numCourses];
3     int graph[numCourses][numCourses];
4     int graphSize[numCourses];
5     for(int i = 0; i < numCourses; i++) {
6         indegree[i] = 0;
7         graphSize[i] = 0;
8     }
9     for(int i = 0; i < prerequisitesSize; i++) {
10        int a = prerequisites[i][0];
11        int b = prerequisites[i][1];
12        graph[b][graphSize[b]++] = a; // b → a
13        indegree[a]++;
14    }
15    int queue[numCourses];
16    int front = 0, rear = 0;
17    for(int i = 0; i < numCourses; i++) {
18        if(indegree[i] == 0)
19            queue[rear++] = i;
20    }
21    int count = 0;
22    while(front < rear) {
23        int course = queue[front++];
24        count++;
25        for(int i = 0; i < graphSize[course]; i++) {
26            int next = graph[course][i];
27            indegree[next]--;
28            if(indegree[next] == 0)
29                queue[rear++] = next;
30        }
31    }
32    return count == numCourses;
33 }
```

Saved Ln 33, Col

☑ Testcase ➤ Test Result

Accepted Runtime: 0 ms

☑ Case 1 ☑ Case 2

801. Minimum swaps to make sequences increasing

```
Code
C v Auto
1 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
2     int keep = 0;    // no swap at i
3     int swap = 1;    // swap at i (1 swap at index 0)
4     for (int i = 1; i < nums1Size; i++) {
5         int newKeep = 1e9;
6         int newSwap = 1e9;
7         // Case 1: sequences already increasing without swap
8         if (nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1]) {
9             newKeep = keep;           // no swap needed
10            newSwap = swap + 1;        // swap continues
11        }
12        // Case 2: cross swap condition
13        if (nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {
14            newKeep = fmin(newKeep, swap); // swap before, not now
15            newSwap = fmin(newSwap, keep + 1); // no swap before, swap now
16        }
17        keep = newKeep;
18        swap = newSwap;
19    }
20    return fmin(keep, swap);
21 }
```

Saved Ln 21, Col

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

287. Find the duplicate number

</> Code

C ▾ 🔒 Auto

```
1 int findDuplicate(int* nums, int n) {
2     int* seen = calloc(n, sizeof(int));
3     for (int i = 0; i < n; i++) {
4         if (seen[nums[i]]) {
5             free(seen);
6             return nums[i];
7         }
8         seen[nums[i]] = 1;
9     }
10    free(seen);
11    return -1;
12 }
```

Saved

Ln 1, Col

☒ Testcase | ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3